[29/10/2026]

Estacionamiento Inteligente

[Bryan Grogan
Xavier Cardenas]

[UCOM418 SISTEMAS OPETATIVOS]
[2026]

Contenido

Descripción 3

Análisis 3

Conclusión 4

Planificación y diseño 5

Descripción

El proyecto **Estacionamiento Inteligente** consiste en desarrollar un sistema que simule y controle el ingreso y salida de vehículos en un estacionamiento con capacidad limitada. El programa permitirá registrar la ocupación de los espacios disponibles y el tiempo de permanencia de cada vehículo.

En esta simulación, cada vehículo estará representado por un hilo de ejecución que podrá intentar ingresar o salir del estacionamiento. El principal problema que se busca resolver es la coordinación de estos hilos cuando varios vehículos intentan acceder al estacionamiento simultáneamente. Para ello, será necesario implementar mecanismos de sincronización que eviten errores en el registro de espacios disponibles y garanticen que no se supere la capacidad máxima establecida.

Análisis

En esta fase de análisis se identifican los componentes clave para la gestión y sincronización del sistema de **Estacionamiento Inteligente.** Las unidades de ejecución corresponden a los hilos que simulan la llegada y salida de cada vehículo, mientras que el recurso principal a gestionar es el número limitado de espacios de aparcamiento disponibles y la estructura de datos que registra su estado. Debido a que múltiples hilos intentan acceder y modificar esta información de manera simultánea, se requiere cooperación y sincronización constante para prevenir condiciones de carrera y garantizar la consistencia de los datos. A continuación, en la Tabla 1, se presenta una comparación detallada entre los modelos de Memoria Compartida y Paso de Mensajes para evaluar cómo se abordaría la implementación, el control de concurrencia y sus respectivas ventajas y desventajas en este escenario.

Tabla 1 Comparación de memoria compartida y paso de mensajes

Aspecto	Memoria Compartida	Paso de Mensajes
Implementación	Los hilos podrían acceder a un objeto Estacionamiento que almacena los espacios disponibles	Los hilos/procesos interactúan a través del envío y recepción de mensajes a un hilo gestor o servidor central que administra el estado del estacionamiento
Ingreso de vehículos	Cada hilo podria consultar y actualizar la disponibilidad, utilizando un lock para evitar modificaciones simultáneas	El hilo del vehículo envía un mensaje de solicitud de ingreso y espera la respuesta del hilo gestor para saber si hay espacio disponible antes de acceder

Salida de vehículos	El hilo libera el espacio ocupado y actualiza el contador compartido de espacios disponibles	El hilo de cada vehículo envía un mensaje de salida al hilo gestor para que actualice los espacios disponibles y libere el lugar
Control de concurrencia	Se utiliza un Lock para evitar modificaciones simultáneas y proteger los datos, y adicionalmente un semaphore para controlar el número limitado de parqueos disponibles restantes	Se gestiona mediante una cola de mensajes en el hilo gestor, procesando las peticiones de ingreso y salida una por una, evitando accesos simultáneos a los datos
Ventajas	Acceso rápido a la información y facilidad para compartir el estado del estacionamiento entre hilos del mismo programa	Evita el uso explícito de locks o variables globales, elimina las condiciones de carrera en el estado compartido y facilita la distribución del sistema en redes o módulos independientes
Desventajas	Requiere sincronización para evitar condiciones de carrera y posibles bloqueos	Mayor sobrecarga por la creación y transferencia de mensajes, y un diseño más complejo para coordinar la comunicación sincrónica y asincrónica
Aplicación al proyecto	Es práctico ya que los vehículos consultarán y modifican constantemente los mismo datos	Permite simular de manera clara el flujo de comunicación real entre las barreras entrada/salida y el sistema central de control del estacionamiento

Conclusión

Tras comparar ambos modelos para el sistema de Estacionamiento Inteligente, se determina que el modelo de Memoria Compartida resulta más conveniente para una implementación basada en hilos dentro de un mismo programa o proceso, ya que ofrece un acceso rápido a la información mediante variables globales protegidas por locks y semáforos.

Planificación y diseño

Esta fase define cómo se administra el recurso compartido del Estacionamiento Inteligente. Según la fase de análisis, se utilizará el modelo de Memoria Compartida: varios hilos que representan vehículos trabajarán sobre un mismo objeto Estacionamiento.

Recurso compartido y su representación

El recurso compartido es el conjunto limitado de espacios del estacionamiento. Cada ingreso reserva un puesto y cada salida lo libera. Como los hilos pueden realizar estas operaciones al mismo tiempo, se necesita sincronización para impedir que dos vehículos ocupen el mismo puesto o que se supere la capacidad maxima.

Se consideran dos representaciones posibles en Python: un entero que almacene los espacios disponibles y un diccionario que registre, por separado, el estado de cada puesto. En ambos casos, la variable capacidad_maxima se definirá al iniciar el programa.

Tabla 2 Comparación de las representaciones del recurso compartido

Aspecto	Tipo básico: entero (int)	Tipo complejo: diccionario (dict)
Representación	espacios_disponibles = capacidad_maxima	espacios = {id_puesto: {estado, vehiculo, hora_ingreso}}
Información registrada	Cantidad total de puestos libres.	Puesto, estado libre/ocupado, vehículo asignado y hora de ingreso.
Ingreso de vehículo	Disminuir el contador en 1 si existe cupo.	Buscar un puesto libre y marcarlo como ocupado con los datos del vehículo.
Salida de vehículo	Aumentar el contador en 1.	Identificar el puesto del vehículo y marcarlo como libre; registrar la salida.
Ventaja principal	Es sencillo y requiere poca memoria.	Permite consultar cada puesto y calcular el tiempo de permanencia.
Limitación	No identifica qué vehículo ocupa cada espacio.	Requiere mayor organización y sincronización de sus actualizaciones.

Clases de Python y relación con los hilos

Se proponen tres clases. Simulador crea un único objeto, Estacionamiento y los hilos de los vehículos; cada instancia de Vehiculo utiliza la misma referencia al Estacionamiento para solicitar un ingreso y registrar su salida. Solo Estacionamiento podrá modificar la estructura compartida, manteniendo el control de las operaciones.

Tabla 3 Clases, atributos y métodos de la solución

Clase	Atributos principales	Responsabilidades y métodos propuestos
Estacionamiento	capacidad_maxima; espacios (dict); lock; cupos (semáforo); historial.	Administrar los puestos. ingresar(id_vehiculo), salir(id_vehiculo), consultar_estado().
Vehiculo	id_vehiculo; estacionamiento (referencia compartida); puesto_asignado; tiempo_permanencia.	Representar un hilo; solicitar ingreso, simular permanencia y solicitar salida mediante ejecutar() o run().
Simulador	estacionamiento; vehiculos (lista de hilos); numero_vehiculos.	Crear el objeto compartido, iniciar y esperar los hilos, y mostrar los resultados de la simulación.

Sección crítica y mecanismo de sincronización

La sección crítica comprende la consulta y modificación del estado de un puesto al ingresar o salir un vehículo. Sin protección, dos hilos podrían elegir el mismo puesto disponible o registrar incorrectamente la ocupación. Se empleará un Lock para que solo un hilo a la vez actualice el diccionario; adicionalmente, un Semaphore inicializado con capacidad_maxima permisos limitará los ingresos simultáneos.

Tabla 4 Operaciones que requieren coordinación

Operación	Riesgo si ocurre simultáneamente	Control propuesto
Ingreso	Dos hilos asignan el mismo puesto o exceden la capacidad máxima.	Obtener un permiso del semáforo; luego, dentro del Lock, buscar y reservar un puesto.
Salida	Se libera un puesto incorrecto o se devuelve dos veces un permiso.	Dentro del Lock, verificar el vehículo y liberar su puesto; después, devolver un permiso.
Consulta del estado	El reporte muestra datos parciales mientras otro hilo modifica la ocupación.	Leer una copia consistente de los puestos bajo el Lock.

El semáforo se solicitará antes de adquirir el Lock, de modo que un vehículo que espere un cupo no bloquee las salidas. La liberación del semáforo solo se hará después de una salida válida. Así, se mantendrá la relación entre la capacidad máxima, los puestos ocupados y los disponibles.

Comparación y selección para la implementación

Aunque el entero permite controlar la cantidad total de espacios disponibles con una implementación simple, no permite identificar el puesto asignado a cada vehículo ni guardar directamente su hora de ingreso. Por esta razón, se selecciona el diccionario como representación principal del recurso compartido. Esta estructura permitirá registrar cada

puesto, identificar al vehículo que lo ocupa y calcular su tiempo de permanencia al salir. La capacidad será configurable y la ocupación se mantendrá consistente.

Implementación y prueba de la concurrencia

En esta fase se desarrolla la solución concurrente en Python para el sistema **Estacionamiento** Inteligente mediante cuatro scripts que evalúan las dos representaciones del recurso compartido (int y dict) bajo escenarios sin sincronización y con mecanismos de control de concurrencia. En todas las pruebas, la clase Simulador instancia un único objeto de estacionamiento (CAPACIDAD = 1) y comparte su referencia entre dos hilos de la clase Vehiculo (V1 y V2) que compiten simultáneamente por el último puesto disponible.

Análisis de la sección crítica

La **sección crítica** comprende las instrucciones secuenciales que consultan, validan y modifican el recurso compartido dentro de los métodos ingresar() y salir():

1. **Consulta y validación:** Lectura de **self.espacios_disponibles > 0** (tipo básico) o recorrido de self.puestos verificando not datos["ocupado"] (tipo complejo).
2. **Actualización:** Decremento/incremento del contador entero o asignación/liberación de las claves "**ocupado**" y "**vehiculo**" en el diccionario.

Inconsistencias provocadas sin sincronización:

- **Tipo básico (01_basico_sin_sincronizacion.py):** Debido al retardo simulado **(time.sleep(0.20)**), V1 y V2 validan simultáneamente que **espacios_disponibles > 0** antes de que alguno actualice el contador. Ambos ingresan, provocando sobrecupo (**ingresos aceptados = 2 para capacidad 1**) y dejando el contador en -1 tras los ingresos.
- **Tipo complejo (03_complejo_sin_sincronizacion.py):** Ambos hilos detectan libre el Puesto #1 al mismo tiempo y sobrescriben la asignación del diccionario (asignaciones entregadas = [1, 1]). Al retirarse, el primer vehículo en salir encuentra que el puesto ya figura a nombre del segundo vehículo, generando un error de consistencia.
-

Justificación del mecanismo de sincronización:

En (02_basico_con_mutex.py y 04_complejo_con_mutex.py) se implementó una estrategia combinada acorde con el diseño propuesto:

- **Mutex (threading.Lock):** Controla la exclusión mutua estricta sobre la sección crítica (consulta y modificación de **espacios_disponibles** o puestos), garantizando que solo un hilo a la vez lea o altere la estructura en memoria. Se

adquiere automáticamente al entrar al bloque with self.lock: dentro de ingresar() y salir(), y se libera al finalizar dicho bloque.

- **Semáforo acotado (threading.BoundedSemaphore(capacidad)):** Controla la cantidad de cupos disponibles. Se adquiere antes de la sección crítica mediante **self.semaforo.acquire(blocking=False)** para rechazar de inmediato a los vehículos cuando el parqueo está lleno, y se libera con **self.semaforo.release()** en el método salir() tras confirmar que el puesto fue desocupado correctamente (o si el ingreso falló).

Tabla 5 Resultados de las pruebas de ejecución concurrente

Escenario Evaluado	Representación	Mecanismo	Configuración	Comportamiento Observado	Estado final del recurso	Consistencia
Escenario 1A	Tipo básico (int)	Sin sincronización	Capacidad: 1 Hilos: 3	Los 3 hilos leen “1” espacio libre e ingresan simultáneamente (sobrecupo)	espacio_disponibles= -1 (valor corrupto)	Inconsistente
Escenario 1B	Tipo básico (int)	Con mutex	Capacidad: 1 Hilos: 3	Vehiculo -1 ingresa, Vehiculo-2 y Vehiculo-3 son rechazados correctamente	espacio_disponibles= -1 (capacidad restaurada)	Consistente
Escenario 2A	Tipo complejo (dict)	Sin sincronización	Capacidad: 1 Hilos: 3	Los 3 hilos asigna el Puesto #1 y sobreescriben el vehiculo registrado	{1:{‘ocupado: False, ‘Vehiculo’: None }} (con colisiones)	Inconsistente
Escenario 2B	Tipo complejo (dict)	Con mutex	Capacidad: 1 Hilos: 3	Solo Vehiculo-1 ocupa y libera el Puesto #1 y los demás son rechazados	{1:{‘ocupado: False, ‘Vehiculo’: None }} (sin colisiones)	Consistente